

1. What is PyTerrier?

IR experimentation framework. Everything is a TRANSFORMER.

```
bm25 = pt.terrier.Retriever(index, wmodel='BM25')
# Nothing runs yet -- bm25 is just an object!
results = bm25.transform(queries_df) # NOW it runs
```

- Input: DataFrame [qid, query]
- Output: DataFrame [qid, docno, rank, score, ...]

(*) **Nothing runs until .transform() or pt.Experiment is called**

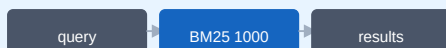
2. Operators (on cribsheet)

Op	Name	What it does
>>	Then	Pipe left output into r
%n	Cutoff	Keep only top-n (bm25
	Union	Combine both result set
&	Intersect	Keep docs in BOTH resul
**	Features	Feature union for LTR t
+	Concat	Append results (used fo

(!) **Always use % BEFORE cross-encoder -- very slow otherwise!**

3. Pattern A: Simple BM25 (baseline)

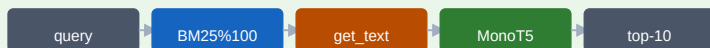
```
bm25 = pt.terrier.Retriever(index,
                             wmodel='BM25', num_results=1000)
```



- Always your baseline in pt.Experiment
- Fast. Suffers from vocabulary mismatch.

4. Pattern B: Re-ranking pipeline

```
reranker = (bm25 % 100
            >> pt.text.get_text(dataset, "text")
            >> MonoT5())
```



(!) **Recall@1000 p-val = NaN when compared to BM25 (see card 9)**

(*) **NDCG@10 improves. get_text() adds text column for MonoT5.**

5. Pattern C: Pseudo-Relevance Feedback

pipe1 -- rewriter sees top-3 docs + index:

```
pipe1 = bm25 % 3 >> Bo1(index) >> bm25
```



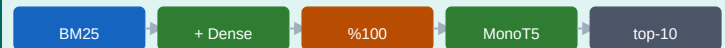
pipe2 -- rewriter sees query ONLY:

```
pipe2 = rewriter2 >> bm25
```



6. Pattern D: Hybrid BM25 + Dense

```
hybrid = ((bm25 + dense) % 100
          >> pt.text.get_text(dataset, "text")
          >> MonoT5())
```



- BM25: exact match (SKUs, brands, product codes)
- Dense: semantic (trainers = sneakers = running shoes)
- (*) **Higher Recall@1000 -- re-ranker gets better candidates**

7. Pattern E: Learning to Rank

```
# ** combines features; LGBMRanker learns weights
features = bm25 ** dense ** pl2_retriever
ltr = bm25 % 1000 >> features >> LGBMRanker()
```



- Each ** component adds a feature column
- (!) **Linear LTR cannot model feature INTERACTIONS (2024 Q3a)**
- (*) **Fix: LambdaMART (tree-based) captures conditional features**

8. rewriter1 vs rewriter2 -- 2025 Q1c

In pipeline: bm25 % 3 >> rewriter1(index) >> bm25

rewriter1: sees top-3 DOCS + index (pseudo-relevance feedback)

- Bo1 (DFR), RM3 -- add terms rare in collection, common in top-3

In pipeline: rewriter2 >> bm25

rewriter2: sees ONLY the original query -- no docs, no index

- Thesaurus (WordNet), Word2Vec -- runs before ANY retrieval

9. "i-phone" tokenisation -- 2025 Q3b(i)

Problem:

- Tokeniser splits "i-phone" -> ["i", "phone"]
- "i" removed as stopword; query = just "phone"
- Docs indexed as "iphone" NEVER match query "phone"

Fix (lexical only, no dense needed):

- Modify tokeniser: treat "i-phone" as single token "iphone"
- Index "iphone" and "i-phone" as same term in vocabulary

10. pt.Experiment -- full template

```
pt.Experiment(
    [bm25, reranker, hybrid],
    topics,           # queries DataFrame
    qrels,            # relevance judgements
    eval_metrics=["ndcg_cut_10",
                  "map", "recall_1000"],
    names=["BM25", "Reranker", "Hybrid"],
    baseline=0        # paired t-test vs sys 0
)
```

11. Metric choices and justification

Metric	What it measures	Justify as
ndcg_cut_10	Top-10 quality (graded)	Primary user-facing met
map	Full ranked list qualit	Ensures deeper ranks no
recall_1000	First-stage coverage	Tests if hybrid > BM25
P_10	Precision at rank 10	Binary relevance tasks
recip_rank	Rank of 1st relevant do	Navigational/known-item

baseline=0: enables auto paired t-test. p<0.05 = significant.

12. NaN p-value -- EXAM FAVOURITE (2024 Q3c)

```
first_stage = bm25
reranker = bm25 % 1000 >> get_text >> MonoT5()
# Recall@1000 p-value = NaN. Why?
```

(!) MonoT5 only RE-ORDERS -- cannot retrieve NEW documents

Why NaN -- step by step:

- Same 1000 candidates for BOTH systems on every query
- Recall@1000 differences = [0, 0, 0, ... 0]
- Std dev = 0 -> t = mean/std = 0/0 = NaN

(*) Not a bug. Fix: dense retrieval, hybrid, or SPLADE

13. 2 experiments -- exam template

```
sparse = lexical >> reranker
hybrid = (lexical + dense) >> reranker

# Exp 1: does dense help beyond BM25?
pt.Experiment([sparse, hybrid], topics, qrels,
    eval_metrics=["ndcg_cut_10", "map",
                  "recall_1000"],
    names=["Sparse", "Hybrid"], baseline=0)

# Exp 2: does BM25 help beyond dense?
dense_r = dense >> reranker
pt.Experiment([dense_r, hybrid], topics, qrels,
    eval_metrics=["ndcg_cut_10", "map"],
    baseline=0)
```

(*) Exp1 tests dense value. Exp2 tests lexical value. Together validate hybrid.

14. Why hybrid? -- 2025 Q3b(iii)

BM25: Exact match -- SKUs, brand names, product codes

Dense: Semantic match -- trainers/sneakers/running shoes

Both: Higher Recall@1000 -- covers failure modes of each

- Re-ranker gets better candidate pool -> better final top-10

(*) Always state NDCG@10 + Recall@1000 as dual justification

15. 2025 Q3b(iv) -- Hybrid pipeline (2 marks)

Q: Design hybrid pipeline. Reranker on all retrieved docs.

```
# (A + B) = union of A and B results
# >> reranker = cross-encoder on ALL union docs
hybrid_pipeline = (lexical + dense) >> reranker
```

(*) Key: (A + B) >> C means union then pipe whole set to C

16. 2025 Q3b(ii) -- LTR for revenue

Q: Rank higher-margin products higher. Use LTR.

```
features = (bm25 ** dense
            ** profit_margin_transformer)
ltr = bm25 % 1000 >> features >> LGBMRanker()
```

- Add profit_margin as query-independent LTR feature
- LambdaMART learns optimal weight for margin vs relevance

(*) Training labels must balance relevance AND business KPIs

17. 2025 Q3b(v) -- Full experiment (7 marks)

```
sparse = lexical >> reranker
hybrid = (lexical+dense) >> reranker
dense_r = dense >> reranker

# Does dense improve over BM25 alone?
pt.Experiment([sparse, hybrid], topics, qrels,
              eval_metrics=["ndcg_cut_10", "map", "recall_1000"],
              names=["Sparse", "Hybrid"], baseline=0)

# Does BM25 improve over dense alone?
pt.Experiment([dense_r, hybrid], topics, qrels,
              eval_metrics=["ndcg_cut_10", "map"], baseline=0)
```

(*) Justify: NDCG@10=user quality, MAP=full list, R@1000=pool size

18. Quick Reference Card

Task	Write this pipeline
Simple BM25	bm25
Re-rank top 100	bm25 % 100 >> get_text >> MonoT5()
PRF expansion	bm25 % 3 >> Bo1(index) >> bm25
Query-only expand	rewriter >> bm25
Hybrid first stage	(bm25 + dense) >> reranker
LTR features	bm25 ** dense ** pl2
Full hybrid pipeline	(bm25+dense) % 100 >> get_text >>

(!) Recall@1000 NaN = re-ranker did not change candidate set

(*) baseline=0 in pt.Experiment enables auto paired t-test

19. Exam strategy for PyTerrier questions

Reading unfamiliar pipeline code:

- >> reads left to right: first stage >> transformation >> reranker
- % cuts the result set: bm25 % 100 = keep only top 100
- + or | = hybrid union of two first-stage retrievers

Writing pipeline code:

- Start with bm25 as first stage unless told to use dense
- Add % cutoff before any expensive transformer
- Add get_text() before any neural re-ranker

Designing experiments:

- Always justify each metric in 1 sentence
- baseline=0 enables t-test -- always include it
- If asked for 2 experiments: ablation (A vs hybrid, B vs hybrid)